CURSO 2026–2027

Instalación y herramientas

Preparar, comprobar y recuperar el laboratorio

José Picón · CIPFP Cheste

Preparar el puesto de trabajo · Curso 2026–2027

Vamos a preparar un entorno que podamos repetir y recuperar. Primero comprobaremos el puesto, después el material y, por último, la práctica concreta. Cada comprobación produce una evidencia: un programa instalado todavía no demuestra que el ejercicio funcione.

1. Qué necesitas y dónde se ejecuta cada cosa

Elemento	Función	Lugar
Windows del aula	Anfitrión de la máquina virtual	PC físico
VirtualBox	Ejecuta la máquina de prácticas	Windows
Kali Linux	Terminal, herramientas y objetivos locales	Máquina virtual
Docker Engine + Compose	Ejecutan aplicaciones del laboratorio	Dentro de Kali
Python y entornos virtuales	Scripts, pruebas y análisis de datos	Dentro de Kali
SOC	Recibe eventos y permite investigar	Servidor Linux o VM dedicada del profesor
Paquete de alumnado	Apuntes, fichas, datos y scripts	`~/xest-lab/2026-2027` dentro de Kali

Los comandos de esta guía se ejecutan en la **terminal de Kali**, salvo que el paso diga Windows o servidor. `localhost` siempre significa el equipo o la VM donde estás ejecutando el navegador/comando. El navegador de Windows no comparte el localhost de Kali.

Como punto de partida para el laboratorio básico, reserva 4 vCPU, 8 GB de RAM y 60 GB de disco para Kali si el anfitrión puede asumirlo. Es una propuesta de preparación, no una medición del aula. Con menos memoria, arranca solo los servicios de la sesión. El SOC completo necesita recursos aparte; no lo arranques junto a todos los objetivos en cada puesto.

2. Importar Kali en Windows

1. Utiliza la máquina del aula preparada por el profesor. Registra el nombre de la imagen, fecha, arquitectura y hash SHA-256.

2. Si recibes una `.ova`, abre VirtualBox → **Importar servicio virtualizado** y selecciona el fichero. Si recibes la imagen oficial comprimida `.7z`, extráela y utiliza **Añadir** sobre el fichero `.vbox`: no se importa como OVA.
3. Revisa RAM, CPU y carpeta de almacenamiento antes de iniciar. La imagen debe corresponder a la arquitectura del equipo: la imagen amd64 del aula no es una VM ARM para un Mac Apple Silicon.
4. Arranca Kali con las credenciales facilitadas. En la imagen oficial, cambia las credenciales iniciales tras el primer acceso; no publiques las de la plantilla del aula.
5. Comprueba teclado, pantalla y terminal. Ejecuta `uname -m`, `python3 --version` y `df -h` y guarda la salida.
6. Apaga Kali y crea una instantánea llamada `base-curso-validada` cuando las comprobaciones de esta guía estén completas. Una instantánea no sustituye la copia de tus entregas fuera de la VM.

Fuente de la distinción `.ova`/`.vbox`: documentación oficial de Kali para VirtualBox.

3. Red del laboratorio

Para prácticas web locales, el navegador y las herramientas funcionan dentro de la misma Kali y los puertos se publican en `127.0.0.1`. Así no necesitan estar expuestos a la LAN.

Las prácticas de interceptación, directorio activo o sensores requieren la topología específica de su ficha. Utiliza una **red interna de VirtualBox** con las VMs participantes, o una red de laboratorio aislada configurada por el profesor. La red interna de VirtualBox no conecta automáticamente con el servidor físico ni con un switch externo.

Para recibir el material, usa USB/carpeta compartida o una conexión aprobada al servidor de aula. Si se utiliza adaptador puente, debe estar conectado a la red aislada de prácticas y verificarse su alcance. NAT se utiliza para preparar actualizaciones con Internet; no da visibilidad automática al sensor sobre otros equipos.

Anota IP, máscara, puerta de enlace y nombre de interfaz:

```
ip -br addr
ip route
```

No copies la IP de un ejemplo como si fuera la asignada a tu equipo. Antes de una práctica activa verifica el objetivo exacto del ROE.

4. Instalar la base con Internet (preparación de la VM)

Este apartado se realiza en la preparación, no en el aula desconectada. En una Kali limpia se puede usar su repositorio para Docker:

```
sudo apt update
sudo apt install -y docker.io docker-compose-v2 python3 python3-venv python3-pip git curl jq unzip
sudo systemctl enable --now docker
sudo usermod -aG docker "$USER"
```

Cierra sesión y vuelve a entrar para cargar el grupo. Después:

```
docker version
docker compose version
docker info
```

Debe aparecer información del cliente y del servidor Docker, y Compose debe reconocer `docker compose`. Si tu edición de Kali no ofrece `docker-compose-v2`, consulta los paquetes disponibles con `apt-cache search docker-compose`; prepara una edición compatible o instala el plugin desde la documentación oficial. No mezcles a ciegas paquetes `docker.io` con `docker-ce`.

Errores habituales:

Error	Qué comprobar	Cómo continuar
Permission denied en docker.sock	`id -nG`	Cierra sesión y entra otra vez; verifica que perteneces al grupo docker
Cannot connect to Docker daemon	`systemctl status docker`	Arranca el servicio y vuelve a ejecutar `docker info`
compose no es un comando	`docker compose version`	Falta Compose v2 en esa máquina
No space left on device	`df -h` y `docker system df`	Amplía espacio o revisa imágenes prescindibles; conserva las de las prácticas offline
Exec format error	`uname -m` y arquitectura de la imagen/binario	Usa artefactos para la arquitectura de la VM

El grupo docker permite operaciones privilegiadas sobre el equipo; se utiliza en la VM de prácticas. Instalación Docker en Kali y alternativa oficial Docker Engine para Debian y derivados.

5. Copiar y comprobar el material sin Internet

1. Copia el paquete **ALUMNADO** completo; conserva su estructura de carpetas.
2. Coloca la carpeta `2026-2027` dentro de `~/xest-lab/`. No la mezcles con una entrega antigua.
3. Abre `INICIO.html` en el navegador: es un índice local y no requiere servidor ni cuenta.
4. Comprueba los archivos con el verificador incluido:

```
cd ~/xest-lab/2026-2027
python3 comun/verificar-material.py
```

El resultado debe mostrar cero archivos ausentes o modificados respecto a la distribución. Ejecuta esta comprobación **antes** de editar las prácticas. Copia los ejercicios a una carpeta de trabajo si quieres

conservar intactos los originales.

6. Python sin romper los paquetes de Kali

Cada práctica con dependencias usa un entorno virtual. No instales paquetes del curso en el Python del sistema con `sudo pip`.

Con Internet, durante la preparación:

```
python3 -m venv ~/venvs/curso
source ~/venvs/curso/bin/activate
python -m pip install pytest pytest-cov coverage flask argon2-cffi requests
python -m pip freeze > ~/venvs/curso/versiones.txt
```

Sin Internet, el profesor debe haber preparado las ruedas para **la misma versión de Python, sistema y arquitectura**:

```
python3 -m venv ~/venvs/curso
source ~/venvs/curso/bin/activate
python -m pip install --no-index --find-links ~/xest-repo/wheels -r ~/xest-repo/requirements-
lock.txt
```

Comprueba `python -c "import pytest, flask, argon2, requests; print('Importaciones OK')"`. Si falta una rueda, no continúes fingiendo una instalación completa: utiliza la VM preparada o completa el repositorio en un equipo con Internet. Un `pip freeze` de macOS no es un repositorio offline para Kali.

7. Cargar las imágenes y arrancar objetivos

El paquete de documentos no incluye las imágenes de varios gigabytes. El profesor proporciona una VM preparada o un repositorio de imágenes exportado tras validar la arquitectura del aula.

```
# Repite para cada .tar recibido y verificado:
docker load -i ~/xest-repo/images/objetivos.tar
cd ~/xest-lab/2026-2027/comun/entornos/objetivos
docker compose config --quiet
docker compose images
```

Arranca únicamente lo necesario:

```
# PPS UT3: portal de referencia
docker compose up -d --pull never portal-soc
curl -fS --connect-timeout 3 --max-time 10 http://127.0.0.1:8080/
```

```
# Prácticas web: Juice Shop y DVWA (incluye su base de datos)
docker compose up -d --pull never juice-shop dvwa
```

`--pull never` evita depender de una descarga en clase. Si la imagen falta, Docker debe avisar: cargarla es un requisito, no un paso opcional. `docker compose config` comprueba la configuración, pero no demuestra que arranque ni que la aplicación funcione.

Servicio	URL en el navegador de Kali	Comprobación funcional
Portal SOC de referencia	`http://127.0.0.1:8080`	Página visible y contenido reconocible
Juice Shop	`http://127.0.0.1:3000`	Catálogo carga; registro/login con cuenta ficticia
DVWA	`http://127.0.0.1:8081/setup.php`	Crear/resetear BD solo al iniciar el laboratorio; después login
Vuln-web histórico	`http://127.0.0.1:8082`	Solo para la ficha que lo requiera
bWAPP histórico	`http://127.0.0.1:8083`	Inicializar siguiendo la imagen; verificar login y ejercicio

DVWA necesita su servicio de base de datos; esta edición lo incluye. El nivel de seguridad se comprueba en la interfaz de DVWA y se registra en cada evidencia. No se da por aplicado por una variable de entorno del compose. Consulta DVWA oficial.

Si una aplicación devuelve 200 pero no permite entrar o completar la operación, aún no está validada. Consulta `docker compose ps` y `docker compose logs --tail=100 NOMBRE-SERVICIO`. No publiques logs que contengan claves.

8. Nombres de laboratorio y puertos

Las fichas usan nombres como `juice.xest.local` o `portal-soc.xest.local` en lugar de direcciones. Así el mismo comando sirve tanto si el objetivo está en tu puesto como si está en un servidor del aula: lo único que cambia es a qué dirección resuelve el nombre.

Dónde está el servicio	Fichero de nombres	Dirección
Objetivos web (Juice Shop, DVWA, portal, bWAPP) en tu Kali — **modo por defecto**	`entornos/objetivos/hosts-puesto`	`127.0.0.1`
SOC (Wazuh, TheHive, MISP) y objetivos servidos por el profesor	`entornos/objetivos/hosts-aula` o DNS del laboratorio	IP de la VLAN que indique el profesor

Añade las líneas solo si el nombre aún no existe, y nunca mezcles las dos versiones del mismo nombre:

```
getent hosts juice.xest.local || sudo tee -a /etc/hosts < comun/entornos/objetivos/hosts-puesto
getent hosts portal-soc.xest.local      # debe mostrar 127.0.0.1 en modo puesto
curl -I --max-time 10 http://portal-soc.xest.local:8080/
```

Si `curl` responde `Connection refused`, el nombre resuelve pero el contenedor no está levantado (`docker compose ps`). Si responde `Could not resolve host`, falta la línea en `/etc/hosts`. Si tarda y acaba en `timed out`, el nombre apunta a una IP del aula que no existe en tu red: revisa que no hayas copiado `hosts-aula` para los objetivos locales.

Varios laboratorios usan el puerto 8080 (portal, proxy, mitmproxy). No los arranques simultáneamente: detén el servicio anterior o aplica el puerto alternativo previsto en la ficha. Confirma con `ss -ltnp`.

9. Cómo añadir herramientas por necesidad

Antes de instalar, identifica la ficha y el resultado que necesitas. El catálogo histórico contiene alternativas y recursos online; no todos son requisitos del curso ni funcionan offline. La guía de herramientas explica los grupos y las pruebas de aceptación.

- Paquete de Kali: `apt-cache policy NOMBRE`, después `sudo apt install NOMBRE` en preparación.
- Aplicación Python: usa `pipx` o un entorno virtual y el proyecto oficial; no deduzcas el nombre del paquete a partir del nombre comercial.
- Binario Go: compila la versión elegida en una máquina compatible y conserva `go version -m RUTA_BINARIO`.
- Contenedor: registra etiqueta, digest, arquitectura, puertos, volúmenes y prueba funcional.
- Servicio con API: registra cuenta/plan, permisos y límites. No lo conviertas en dependencia de una clase sin Internet.

Para preparar una OVA, instala las herramientas de cada bloque, abre las aplicaciones gráficas al menos una vez, precarga datos/reglas y reinicia sin conexión antes de exportar. Un verificador de nombres de ejecutables no sustituye ese ensayo.

10. Cierre de sesión y recuperación

Guarda informe, comandos, versiones, salidas y hashes antes de detener el entorno. `docker compose stop` permite parar; `docker compose down` elimina contenedores y redes del proyecto, pero normalmente conserva volúmenes. No utilices `down -v` como cierre rutinario: elimina datos persistentes.

Si necesitas repetir desde cero, usa la instantánea inicial o el procedimiento de reinicio de la práctica, conservando fuera las evidencias. Anota qué has restaurado: código, base de datos, VM o todos ellos.

Herramientas del curso: para qué sirven y cómo comprobarlas

La herramienta se elige después de definir la pregunta. En cada práctica registra versión, entrada, opciones, resultado y limitaciones. Las alternativas de los arsenales de HE son recursos de ampliación: no necesitas instalar todas para completar el itinerario básico.

1. Base común

Herramienta	Uso en el curso	Instalación en la VM de preparación	Primera comprobación
Git	Historial, ramas y evidencias de cambios	`sudo apt install git`	`git --version`; crear repo de prueba y comprobar un commit
Python	Scripts y análisis local	`sudo apt install python3 python3-venv`	`python3 --version`; ejecutar verificador del material
curl	Peticiones y cabeceras HTTP	`sudo apt install curl`	`curl -I --max-time 10 http://127.0.0.1:8080/` con portal arrancado
jq	Consultar JSON	`sudo apt install jq`	`printf '{"ok":true}\n' | jq -e '.ok'`
Docker/Compose	Objetivos y servicios	Guía de instalación	Cliente, daemon, compose y respuesta funcional del servicio
Navegador	Aplicaciones, HTTP y material	Incluido en la VM	Abrir índice local y pestaña Red de las herramientas de desarrollo

En el aula sin Internet estas instalaciones deben estar ya hechas o llevar todos los paquetes y dependencias. `apt --download-only` sobre una máquina que ya tiene paquetes instalados **no garantiza** un repositorio suficiente para otra máquina limpia.

2. PPS: desarrollo y pruebas

pytest y coverage. Se instalan dentro del entorno virtual de la práctica con `python -m pip install pytest coverage pytest-cov`. Desde `PPS/UT1/laboratorio/miniproyecto` ejecuta `python -m pytest -q`, después `python -m coverage run -m pytest` y `python -m coverage report -m`. Debes distinguir número de pruebas, resultado y líneas no ejercitadas. Si pytest no encuentra tests, verifica el directorio de trabajo; cero tests no es una suite aprobada.

Flask. Es el framework de los pequeños portales del curso. Instala las dependencias del `requirements.txt` de la práctica dentro de su venv. Arranca con el comando de su README y abre la URL indicada. Si un import falla, comprueba `which python` y `python -m pip show flask`; suele haberse instalado en otro intérprete. Las aplicaciones vulnerables se mantienen en laboratorio.

Argon2id. En Python se usa `argon2-cffi`, importado como `argon2`. Instala en venv y ejecuta las pruebas de migración de la UT5. La comprobación no es solo que el hash empiece por un prefijo: verifica alta, autenticación incorrecta y migración transparente sin perder acceso.

C, Java y Go. Se utilizan para comparar modelos de ejecución y gestión de memoria. Instala `build-essential`, `default-jdk` y `golang-go` en la VM preparada. Comprueba `gcc --version`, `java -version`, `javac -version` y `go version`. Ejecuta los ejemplos únicamente en el laboratorio y separa fallo deliberado del ejercicio de error de compilación.

strace y seccomp. `strace` muestra llamadas al sistema; seccomp filtra las que puede realizar un proceso. Instala `strace` en la imagen que utiliza la ficha. La prueba de aceptación compara la misma llamada y los mismos privilegios con y sin perfil. Un error por falta de capacidad no acredita por sí solo que seccomp la haya bloqueado.

3. PPS: análisis y canalización

Control	Herramienta	Preparación	Aceptación
Código	Semgrep	Venv/pipx y reglas locales del laboratorio; registrar versión	Detecta el ejemplo vulnerable y devuelve el código de salida exigido por CI
Dependencias e imágenes	Trivy	Binario oficial compatible y bases precargadas	Identifica imagen/lockfile, fecha de base y hallazgo conocido
Secretos	Gitleaks	Release oficial compatible con arquitectura	Detecta marcador sintético configurado y permite el caso limpio
Infraestructura como código	Checkov	Entorno Python aislado	Señala el control incumplido del `infra/main.tf` de prueba

Control	Herramienta	Preparación	Aceptación
Proxy/reverse proxy	nginx	Imagen fijada en compose	Configuración válida y operación HTTP/TLS antes/después
TLS	testssl.sh	Copia del repositorio/release oficial preparado	Informe del endpoint TLS exacto, no de un puerto HTTP
Host	Lynis	Paquete de la distribución	Informe del sistema analizado con permisos y alcance registrados

Las fichas de UT7 contienen los comandos del ejercicio. Antes de copiarlos consulta `--help` en la versión instalada: algunas interfaces cambian. En preparación descarga también reglas, acciones de CI y bases; no basta guardar el ejecutable. Si falta la base de Trivy, la ausencia de resultados no significa ausencia de vulnerabilidades.

Referencias de instalación: Semgrep, Trivy, Gitleaks, Checkov, testssl.sh, Lynis. Son enlaces de preparación; el aula utiliza la versión ensayada y registrada, no una actualización improvisada.

4. PPS: móvil

jadx convierte bytecode Android en una representación de código legible. Prepara la release oficial o el paquete de Kali y comprueba `jadx --version`. Usa un APK del laboratorio y registra su hash. La carpeta `salida_jadx` incluida permite el análisis de fuentes sin APK, pero se debe declarar esa modalidad.

MobSF automatiza parte del análisis móvil. Prepara su contenedor o despliegue oficial, con sus dependencias e imagen almacenadas antes de entrar en el aula. Comprueba que permite subir el APK y generar un informe. El JSON de ejemplo de UT6 es material de análisis, no prueba de haber ejecutado MobSF.

mitmproxy permite observar tráfico HTTP y TLS de los clientes del laboratorio. Instálalo en un entorno aislado o mediante el paquete de la VM; comprueba `mitmproxy --version`. Configura el proxy en el cliente y, solo en el entorno de prueba, la CA necesaria. La aceptación consiste en ver una petición propia; para pinning, documenta el rechazo esperado y su causa. No desactives la verificación TLS como solución general.

5. HE: OSINT y reconocimiento

Empieza con el kit local de HE UT5: usa Python, Git, CSV y el navegador. No requiere buscar a personas ni disponer de cuentas externas. El cuaderno de octubre explica las fuentes, el grafo, los descartes y la entrega.

Familia	Herramientas disponibles en el catálogo	Qué aportan	Dependencia importante
Metadatos	ExifTool	Campos de un fichero	Disponer del original; los metadatos pueden ser modificados
Alias	Sherlock, Maigret	Coincidencias por nombre de usuario	Acceso a sitios; coincidencia no acredita identidad
Correo/dominio	theHarvester, subfinder, Amass	Referencias, subdominios y fuentes	APIs, cuotas y conectividad según módulos
Agregación	SpiderFoot, Recon-ng	Organizar y cruzar fuentes	Revisar cada módulo: puede ser activo y requerir API
Web histórica	Wayback, gau	URLs observadas históricamente	Archivos externos; resultado histórico no acredita estado actual
Certificados	crt.sh y capturas locales	Nombres incluidos en certificados	Fecha y alcance de la fuente; certificado no prueba servicio accesible
Exposición	HIBP y simulaciones	Referencias a incidentes	Cuenta/servicio y datos autorizados; aula usa simulación

Para ExifTool en Kali prepara `libimage-exiftool-perl`; comprueba `exiftool -ver` y analiza un fichero didáctico. Para las aplicaciones Python usa el proyecto oficial y pipx/venv; no copies una lista de nombres a pip asumiendo que son todos paquetes válidos. Los recursos y enlaces de cada familia están en `HE/recursos/arsenal-osint.md` y en el cuaderno revisado de octubre.

Nmap enumera puertos y servicios del objetivo permitido. Prepara `nmap` desde Kali y comprueba `nmap --version`. En el portal local, un ensayo acotado es `nmap -sT -p 8080 127.0.0.1`; el puerto debe coincidir con el servicio arrancado. Un puerto cerrado no acredita que el equipo no exista. HE-E10 exige documentar opciones y alcance.

ffuf/feroxbuster prueban rutas a partir de diccionarios; **httpx de ProjectDiscovery** sondea servicios HTTP; **Nuclei** aplica plantillas. Cada uno produce candidatos a revisar. El binario `httpx` de una biblioteca Python no es necesariamente el de ProjectDiscovery. Comprueba fabricante/versión y una petición local antes de clase. Las plantillas y wordlists deben estar descargadas.

6. HE: auditoría web

Burp Suite Community / OWASP ZAP. Prepara el instalador oficial o el paquete de Kali y abre la interfaz antes de exportar la VM. Configura el navegador de prácticas para su proxy, comprueba una petición al objetivo local y desactiva el proxy al finalizar. Para HTTPS, instala la CA solo en ese perfil de navegador. Identifica exactamente qué funciones están disponibles en la edición instalada.

sqlmap, Nikto, Nuclei y herramientas XSS/JWT amplían pruebas; no sustituyen el examen manual. Usa las opciones concretas de la ficha, objetivo autorizado y límites previstos. Una lista de alertas sin peticiones verificables no constituye un informe. El JWT puede decodificarse sin conocer el secreto: eso no demuestra que su firma sea válida ni que pueda modificarse sin rechazo.

Selenium automatiza navegación y regresiones. Necesita navegador, controlador compatible y dependencias Python. Prepara todo con Internet y comprueba una navegación local tras desconectar. Las descargas automáticas del controlador pueden fallar en clase; deja el controlador validado incluido en la VM.

7. HE: infraestructura, credenciales y radio

Herramientas	Función docente	Preparación y criterio de aceptación
Metasploit / searchsploit	Relacionar fallo, condiciones y demostración de laboratorio	Paquetes de Kali, base local disponible y objetivo vulnerable de la ficha; registrar sesión y efecto
Bettercap / Responder	Observar intercepción y autenticación en segmento didáctico	Interfaces y VMs del ejercicio; captura de referencia y contramedida contrastada
Impacket / NetExec	Examinar protocolos y permisos del dominio de pruebas	Entorno de instalación oficial aislado; usar cuentas ficticias y verificar una operación autorizada
BloodHound CE	Analizar relaciones y caminos de acceso	Despliegue propio y dataset de laboratorio importado; grafo con nodos y relaciones esperados
John / Hashcat	Contrastar resistencia de credenciales sintéticas	Formato/hash correcto, backend CPU/GPU compatible y diccionario de la práctica; registrar resultado y tiempo
Aircrack-ng / hcxtools	Analizar capturas Wi-Fi	Adaptador/driver compatibles y AP propio, o captura suministrada marcada como tal

Herramientas	Función docente	Preparación y criterio de aceptación
Proxmark / URH	Prácticas RFID y radio	Hardware compatible, firmware y dispositivo didáctico; verificar lectura/recepción antes de plantear emisión
Herramientas de túnel	Comprender segmentación y rutas	Solo topología del laboratorio; comparar conectividad antes/después y retirar el túnel al terminar

En máquinas virtuales, el acceso USB, el modo monitor y la GPU no se deducen de que un paquete esté instalado. Si falta hardware, utiliza los datasets y registra la modalidad. No des por cumplida una evidencia de captura/emisión real con un análisis de fichero.

Los catálogos de HE conservan C2 y otros recursos avanzados como ampliación. No se instalan como parte del arranque básico ni se incluyen en la prueba general de todas las unidades.

8. IC: servicios y análisis

Wazuh reúne manager, indexer y dashboard; los agentes envían eventos. Utiliza el despliegue recuperado en `SOC/plataforma`, su `.env.example`, hashes internos y certificados. No basta ejecutar compose: revisa la guía SOC de esta edición. La aceptación es un evento conocido convertido en alerta, con agente y hora correctos.

Suricata analiza el tráfico visible en una interfaz. Antes de arrancar anota interfaz, red monitorizada y reglas. Valida configuración y un caso positivo/negativo. Sin mirror/SPAN o topología equivalente no ve por arte de magia todo el tráfico del aula.

TheHive/Cortex organizan casos y análisis de observables. Prepara el stack y sus dependencias, cuentas y permisos. Crea un caso ficticio con tarea y observable para verificarlo. Los analizadores que llaman a servicios externos no funcionarán offline salvo alternativa local configurada.

MISP/STIX permiten estructurar e intercambiar inteligencia. Valida sintaxis JSON, esquema STIX y, por separado, importación y consulta del objeto. No subas indicadores del aula a comunidades reales.

Velociraptor recoge artefactos; **Volatility** analiza memoria adquirida. Prepara versiones compatibles, símbolos y máquinas de prueba. Comprueba un artefacto controlado o una imagen conocida. El curso incluye salidas sintéticas para trabajar el razonamiento cuando no haya adquisición real; deben etiquetarse como tales.

GoPhish/MailHog se usan para simulación local de concienciación. Verifica el circuito con destinatarios ficticios y buzón sumidero. Una métrica de apertura puede proceder de automatismos y necesita interpretación.

9. Registro de instalación que debes conservar

Por herramienta: nombre, versión, fuente oficial, fecha, SO/arquitectura, método de instalación, dependencias, datos/reglas/API, comando de comprobación, prueba funcional, resultado y solución de errores. La plantilla `REGISTRO_ENTORNO.csv` permite recogerlo. Marca como pendiente cualquier prueba que no hayas ejecutado.

Primera sesión práctica: terminal, Git y servicio web

Al terminar podrás explicar qué fichero estás sirviendo, qué cambio has guardado en Git y qué diferencia hay entre imagen y contenedor. Trabajaremos dentro de Kali y conservaremos evidencias de cada paso.

1. Situarte en el sistema

```
pwd
mkdir -p ~/trabajo-curso/arranque
cd ~/trabajo-curso/arranque
pwd
```

`pwd` muestra la carpeta actual; `mkdir -p` crea el directorio si falta; `cd` cambia a él. El segundo `pwd` debe terminar en `/trabajo-curso/arranque`. Si no coincide, corrige la ruta antes de crear archivos.

2. Crear una página mínima

```
cat > index.html <<'HTML'
<!doctype html>
<html lang="es">
<meta charset="utf-8">
<title>Mi laboratorio</title>
<h1>Laboratorio de ciberseguridad</h1>
<p>Primera versión del servicio de prácticas.</p>
</html>
HTML
cat index.html
```

`cat >` guarda el texto en el archivo. El marcador HTML delimita el contenido; no forma parte del archivo. Abre `index.html` desde el navegador para distinguir un documento local de una respuesta servida por HTTP.

3. Registrar el cambio con Git

```
git init
git config user.name "Alumno NN"
git config user.email "alumno-nn@xest.example"
git status
git add index.html
```

```
git commit -m "Crea la página inicial del laboratorio"
git log --oneline -1
```

Cambia NN por tu número de lista. La configuración se aplica solo a este repositorio. `add` prepara el contenido que entrará al commit; `commit` crea un punto del historial. Un commit no implica que hayas subido nada a Internet.

Modifica la frase de la página con el editor y ejecuta `git diff`. Comprueba que aparece exactamente tu cambio; guarda otro commit. Explica qué representa cada identificador del historial.

4. Servir la página por HTTP

Comprueba que la imagen nginx fijada del curso está disponible. Con el fichero de bloqueo o compose proporcionado, el profesor debe haber cargado la imagen antes de clase. Para este ejercicio se utiliza la misma referencia fijada:

```
docker run --rm --name curso-arranque-nn -d \
  -p 127.0.0.1:18090:80 \
  --mount "type=bind,source=$PWD,target=/usr/share/nginx/html,readonly" \
  nginx:1.27-alpine@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10
curl -i --max-time 10 http://127.0.0.1:18090/
```

El contenedor ejecuta nginx. `-p` conecta el puerto local 18090 con el 80 del contenedor. El montaje permite leer tus archivos desde nginx y es de solo lectura dentro del contenedor. `--rm` elimina el contenedor cuando se detiene; no borra tu carpeta de trabajo.

Debes observar una respuesta HTTP satisfactoria y el texto de tu página. Abre la misma URL en el navegador de Kali. Cambia una frase, recarga y explica por qué se ve el cambio sin construir una imagen nueva.

5. Cerrar y justificar

```
docker logs curso-arranque-nn
docker stop curso-arranque-nn
git status
```

Entrega un informe corto con: ruta del proyecto, dos commits, petición/respuesta HTTP, explicación del montaje y diferencia entre abrir un archivo y pedirlo a un servidor. Si el puerto estaba ocupado, documenta cómo lo comprobaste y qué puerto alternativo utilizaste. No sustituyas comandos fallidos por capturas de ejemplo.

NN-MOD-EX · Título de la práctica

Objetivo y alcance

Qué voy a comprobar, activos autorizados y límites. Indica si trabajas con un entorno real de laboratorio, una captura facilitada o datos sintéticos.

Entorno

SO, arquitectura, versiones, referencia de imágenes, herramientas, reglas/datos y estado inicial. Incluye el identificador de la instantánea si la has utilizado.

Procedimiento

Por cada paso: carpeta de trabajo, comando o acción, qué esperabas, qué has observado y archivo que lo acredita. Conserva los comandos como texto además de las capturas.

Resultados

ID	Observación	Evidencia	Interpretación	Limitación
E01				

Verificación

Caso positivo, caso negativo y resultado de la repetición. Si propones una corrección, explica cómo comprobarás que resuelve el fallo sin romper una operación legítima.

Incidencias y decisiones

Problema, diagnóstico, cambio aplicado y prueba de que funcionó. Distingue lo solucionado de lo pendiente.

Cierre

Estado final del laboratorio, limpieza de artefactos del ejercicio y copia de evidencias. Enumera lo que entregas y el hash de los archivos que deban preservarse.

Referencias

Fuente, versión/fecha y parte del trabajo para la que se ha utilizado. Reconoce ayuda y recursos utilizados conforme a las reglas de evaluación del módulo.

Precisiones que aplicaremos durante el curso

Evidencia y alcance

Un hash permite comprobar integridad respecto de una referencia; por sí solo no demuestra autoría, autenticidad, licitud ni toda la cadena de custodia. Conserva procedencia, fechas, responsables y transferencias.

Las salidas sintéticas de Volatility y Velociraptor son material de análisis. No incluyen una imagen `memoria.raw` ni acreditan una adquisición real. La ejecución sobre memoria necesita que el profesor prepare un volcado de laboratorio compatible y los símbolos correspondientes.

Datos personales y autorización

Usaremos por defecto el caso OSINT sintético incluido y no buscaremos sus identidades en Internet. Un dato accesible públicamente no elimina las obligaciones de tratamiento. El consentimiento no es la única base jurídica posible y un documento de alcance no autoriza cualquier conducta. Los artículos 197 bis y 197 ter del Código Penal regulan supuestos distintos; no se debe enseñar que instalar una herramienta de doble uso sea por sí solo delito. Código Penal, texto consolidado del BOE.

Notificación de incidentes

En el ejercicio RGPD distingue brecha de datos, riesgo, alto riesgo y destinatario. La notificación a la autoridad competente debe efectuarse sin dilación indebida y, si es posible, dentro de 72 horas desde que se tiene constancia, salvo que sea improbable que suponga riesgo para derechos y libertades. Documenta también las brechas que no se notifican. La comunicación a afectados sigue su propio criterio de alto riesgo y no se resume en el mismo reloj de 72 horas. AEPD: notificación de brechas, AEPD: comunicación a personas afectadas.

NIS2 se estudia como marco europeo: identifica ámbito, incidente significativo, alerta temprana, notificación y seguimiento. El artículo 23 contempla, con sus condiciones, hitos de 24 y 72 horas y un informe final. Antes de trasladarlo a un caso real hay que verificar transposición española vigente, autoridad y régimen sectorial; esta edición no certifica ese análisis jurídico caso por caso. No se asigna una autoridad solo por ser una entidad pública o privada. Directiva (UE) 2022/2555, BOE.

Interpretar herramientas

La selección de seguridad high en DVWA no equivale a ausencia de vulnerabilidades. Se verifica el código y el comportamiento de la versión del laboratorio. Un JWT decodificado no está por ello validado. Una sonda

de cabeceras no verifica todo ASVS. En ASVS registra edición e identificadores antes de contrastar requisitos. ASVS oficial.

El orden KEV → EPSS → CVSS es la regla simplificada del ejercicio de priorización; una decisión de parcheo real también considera el activo, exposición, dependencias y controles. Los snapshots son didácticos, no una fotografía certificada de amenazas actuales.

Tiempo y coordinación

Se conservan 65 horas de referencia para PPS, 65 para HE y 80 para IC de la programación recuperada. El material original contiene ampliaciones que exceden esos tiempos. Las rutas de esta edición distinguen un núcleo de trabajo del banco ampliado de sesiones. La asignación final del centro y los festivos deben fijarse con su calendario real.

PPS se imparte el martes, IC el miércoles y HE el jueves según el horario guardado. Las evidencias que produce HE el jueves se investigan en IC el miércoles siguiente. HE UT8 requiere el servicio de PPS UT4–UT5; la parte de medición de ruido de HE-E10 requiere un sensor operativo de IC.